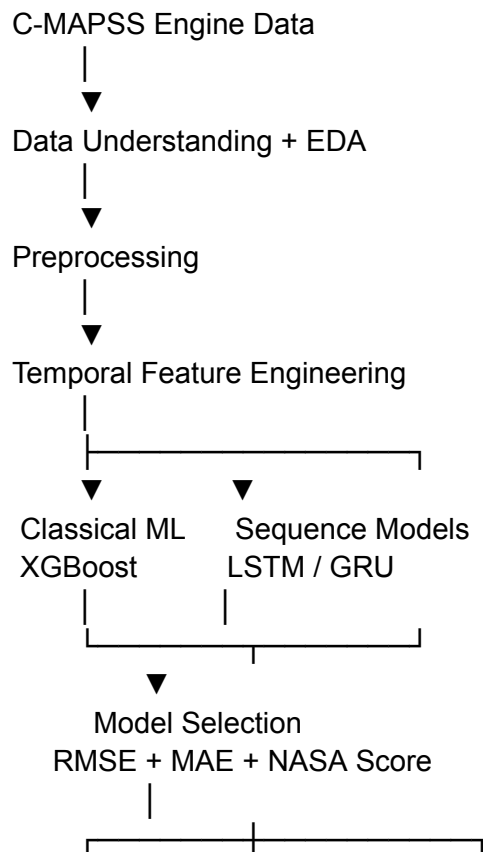
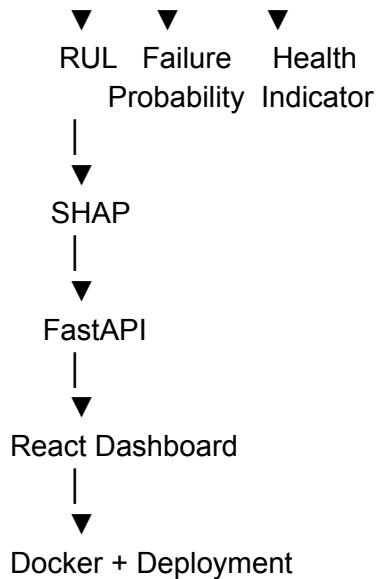


Make uv environment first and install all dependencies there , make proper jupyter notebook for ml pipeline in proper order with markdown brief explanation so that i can run and decide , donot train models yourself just write the code , step by step

0. Final system you are aiming for





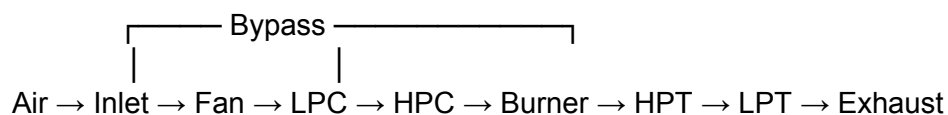
The **MVP** should end with a working deployed system. Transformer, anomaly detection, DTW and what-if analysis come afterward.

PHASE 0 — Project setup and C-MAPSS understanding

Time: 1–2 days

Step 0.1 — Understand the physical system

Before touching ML, understand the turbofan flow path:



The paper describes five rotating components:

Fan → LPC → HPC → HPT → LPT

and models degradation through health-related parameters such as component **flow capacity and efficiency**.

That's important because your sensors are indirect measurements of these underlying degradation processes.

Step 0.2 — Understand the sensors physically

Don't leave your README with only `sensor_1`, `sensor_2`, etc.

The paper identifies the sensor outputs used to create the challenge data.

Symbol	Physical quantity
T2	Fan inlet total temperature
T24	LPC outlet total temperature
T30	HPC outlet total temperature
T50	LPT outlet total temperature
P2	Fan inlet pressure
P15	Bypass duct pressure
P30	HPC outlet pressure
Nf	Physical fan speed
Nc	Physical core speed
epr	Engine pressure ratio
Ps30	HPC outlet static pressure
phi	Fuel flow / Ps30
NRf	Corrected fan speed
NRc	Corrected core speed
BPR	Bypass ratio
farB	Burner fuel-air ratio

htBleed	Bleed enthalpy
Nf_dmd	Demanded fan speed
PCNfR_dmd	Demanded corrected fan speed
W31	HPT coolant bleed
W32	LPT coolant bleed

This mapping is useful when interpreting SHAP later.

Step 0.3 — Understand what degradation means

The paper's central idea is roughly:

Wear
↓
Flow/Efficiency degradation
↓
Thermodynamic engine response changes
↓
Temperature / pressure / speed etc. change
↓
Sensors capture those changes
↓
ML detects degradation
↓
Predict RUL

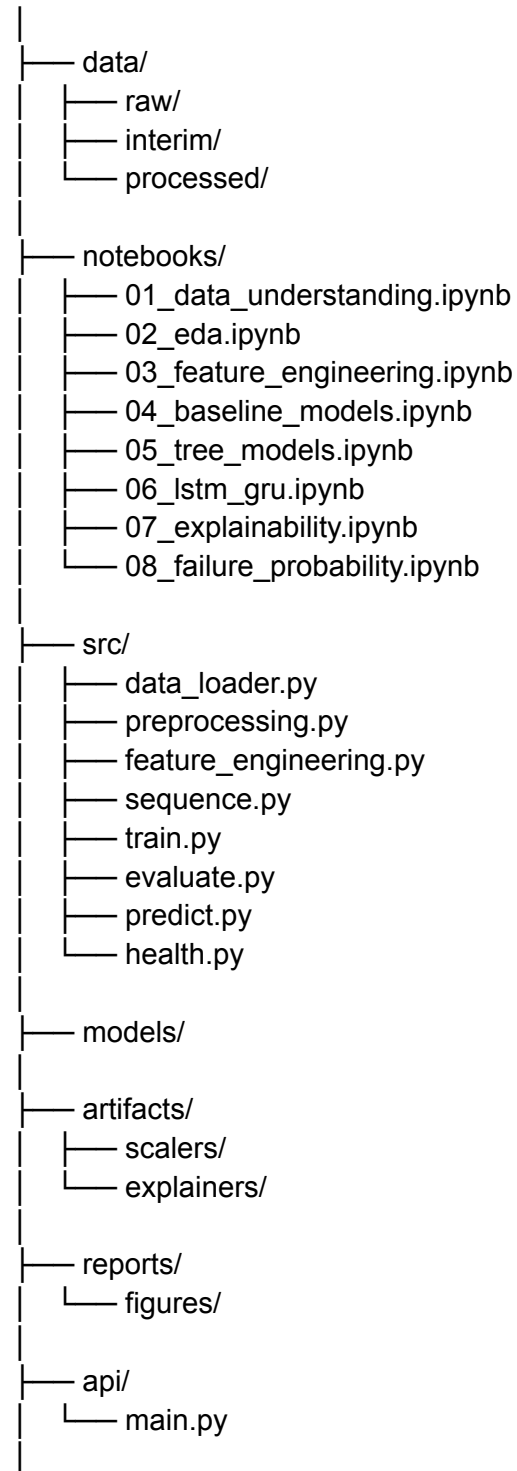
The simulated degradation is physics-inspired and approximately exponential rather than simply linear.

This will become important when analyzing sensor trends.

Step 0.4 — Create the repository

I'd slightly improve your proposed structure:

aircraft-predictive-maintenance/



```
|— frontend/
|
|— tests/
|
|— requirements.txt
|— Dockerfile
|— docker-compose.yml
|— README.md
```

Use **VS Code + Jupyter extension** for notebooks. Keep the whole repository in one place.

PHASE 1 — Data understanding

Time: 2–3 days

Start with **FD001**.

Don't combine FD001–FD004 immediately.

Step 1.1 — Load the dataset

Assign:

```
engine_id
cycle

op_setting_1
op_setting_2
op_setting_3

sensor_1
...
sensor_21
```

Check:

```
df.head()
df.shape
df.info()
df.describe()
```

Calculate:

number of engines
number of observations
cycles per engine
minimum lifetime
maximum lifetime
average lifetime

Step 1.2 — Understand one observation

C-MAPSS is **not ordinary tabular data**.

Engine 1
cycle 1
cycle 2
cycle 3
...
cycle 192

Engine 2
cycle 1
cycle 2
...

Each engine is a separate multivariate time series.

Therefore:

Engine = sequence

Cycle = time

Sensors = measurements

RUL = target

This understanding determines almost everything later.

Step 1.3 — Generate RUL

For training engines:

$$RUL_{i,t} = T_i - t \quad RUL_{i,t} = T_i - t$$

where T_i is the last cycle for engine i .

For example:

Engine failure = cycle 200

Cycle	RUL
-------	-----

1	199
50	150
100	100
150	50
190	10
200	0

Implementation:

```
max_cycles = df.groupby("engine_id")["cycle"].transform("max")
df["RUL"] = max_cycles - df["cycle"]
```

Plot RUL against cycle for several engines to verify it.

Step 1.4 — Understand train vs test carefully

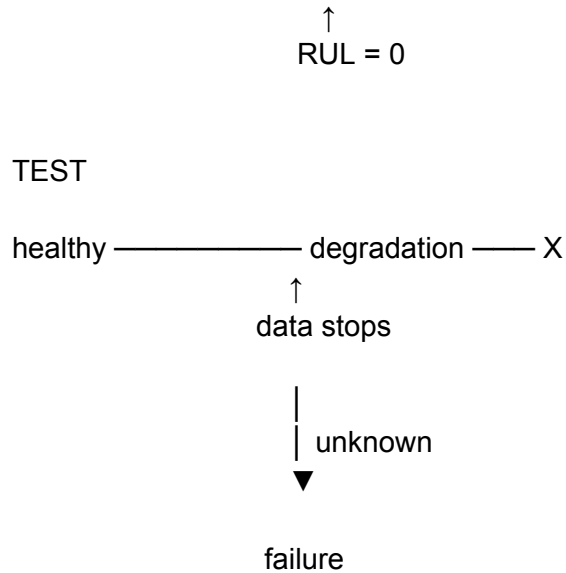
The paper explains an important detail.

Training trajectories run until the failure threshold, whereas test trajectories are **truncated before failure**.

Conceptually:

TRAIN

healthy ————— degradation ————— failure



That is exactly the real-world prognostics problem.

PHASE 2 — Exploratory Data Analysis

Time: 3–4 days

This should be much deeper than `df.describe()`.

Step 2.1 — Data quality

Check:

Missing values

Duplicates

Infinite values

Incorrect types

Number of unique values

Create a summary dataframe.

Step 2.2 — Sensor variance

Calculate:

```
df[sensor_cols].var()
```

Classify sensors as:

constant

near constant

variable

Don't automatically delete anything yet.

Document your reasoning.

Step 2.3 — Sensor distributions

For every sensor:

Histogram

Boxplot

Mean

Std

Min

Max

Variance

You'll start seeing which sensors contain useful information.

Step 2.4 — Degradation plots

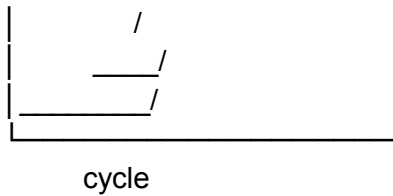
This is one of the most important parts of your project.

Pick several engines.

Plot:

sensor value

|
| /



Do this for multiple sensors.

Look for:

- increasing trends
- decreasing trends
- no trends
- noise
- late-life changes

The paper explains why you should **not expect perfectly monotonic degradation**: process noise and simulated between-flight effects can cause temporary apparent improvements even while the overall engine deteriorates.

That is a strong observation to mention in your notebook.

Step 2.5 — Smooth sensor trajectories

Compare:

- Raw signal
- vs
- Rolling mean
- vs
- EMA

For example:

```
engine["sensor_11"].rolling(10).mean()
```

This helps separate:

- long-term degradation
- from
- short-term noise

The paper explicitly includes process and measurement noise in data generation.

Step 2.6 — Correlation

Calculate:

sensor ↔ RUL

sensor ↔ sensor

operational setting ↔ sensor

Create:

Correlation heatmap

Then:

Top sensors correlated with RUL

Don't select features purely based on correlation.

Temporal nonlinear relationships may not have strong Pearson correlation.

Step 2.7 — Outlier analysis

Use:

IQR

Z-score

Boxplots

But **don't blindly remove outliers**.

In predictive maintenance:

Extreme reading

could be

sensor noise

OR

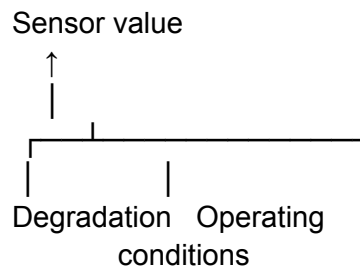
actual degradation

Deleting degradation observations could destroy exactly the information you need.

Step 2.8 — Operating conditions

The paper explains that operational conditions affect the engine response and therefore the measured sensor values.

Conceptually:



This is a major challenge.

Explore:

op_setting vs sensor

op_setting vs RUL

Later, operating-condition clustering becomes useful especially for the more complex C-MAPSS subsets.

PHASE 3 — Preprocessing

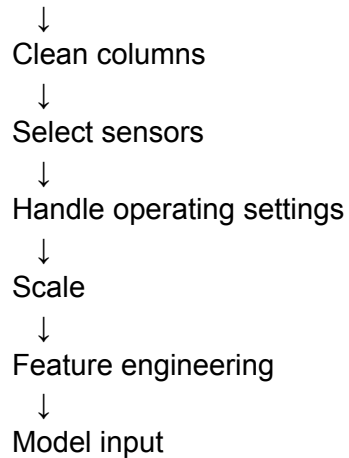
Time: 1–2 days

Now move stable preprocessing code into:

`src/preprocessing.py`

Pipeline:

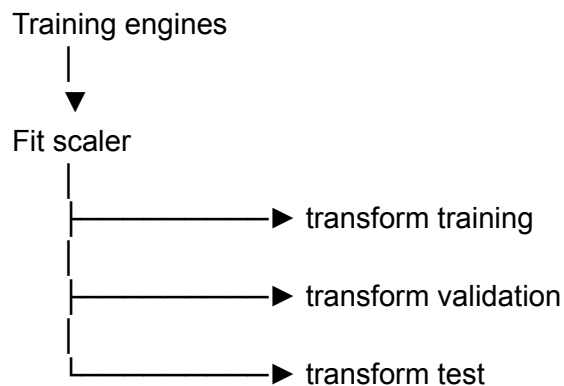
Raw data



Critical issue: data leakage

Do **not** fit a scaler using the entire dataset.

Correct:



Validation/test data must never determine preprocessing parameters.

PHASE 4 — Feature engineering

Time: 3–5 days

This matters mainly for classical ML.

Step 4.1 — Raw features

Start with:

cycle
operational settings
selected sensors

This becomes your simplest baseline.

Step 4.2 — Lag features

For useful sensors:

`sensor(t)`

`sensor(t-1)`

`sensor(t-3)`

`sensor(t-5)`

Example:

```
grouped = df.groupby("engine_id")
```

```
df["s11_lag1"] = grouped["sensor_11"].shift(1)
```

```
df["s11_lag5"] = grouped["sensor_11"].shift(5)
```

Grouping by engine is critical.

Step 4.3 — Rolling statistics

Try windows:

5

10

20

Generate:

rolling mean
rolling std
rolling min
rolling max

For example:

sensor11_mean_5
sensor11_std_5

sensor11_mean_10
sensor11_std_10

Step 4.4 — Rate of change

Very relevant to deterioration:

$$\Delta s_t = s_t - s_{t-1}$$

Create:

sensor_4_diff1
sensor_7_diff1
sensor_11_diff1
...

You can later investigate slopes over windows rather than only one-step differences.

Step 4.5 — EMA

Generate exponential moving averages:

ewm(span=10)

This provides smoothed recent history.

Step 4.6 — Operating-condition features

For appropriate subsets, experiment with:

op_setting_1
op_setting_2
op_setting_3

↓

KMeans

↓

operating_condition_cluster

Then consider condition-aware normalization.

Step 4.7 — Data-driven health indicator

This is a good research-inspired addition.

Remember: the paper has an internal health index based on operational margins, but that health index was **not supplied to challenge participants**.

So don't claim you reconstructed NASA's true health index.

Instead create:

Data-Driven Health Indicator

Possible approach:

Selected normalized degradation sensors

↓

PCA

↓

1-dimensional degradation representation

Plot it against cycle/RUL.

Later you can compare this with an autoencoder-derived health representation.

PHASE 5 — Proper dataset splitting

This is one of the most important technical decisions.

Never do:

```
train_test_split(df)
```

randomly across rows.

You could get:

Engine 14 cycles 1–100 → training

Engine 14 cycles 101–150 → validation

Leakage.

Instead:

TRAIN

Engines A, B, C, D...

VALIDATION

Engines E, F...

TEST

Official C-MAPSS test engines

Use:

```
GroupShuffleSplit
```

or

```
GroupKFold
```

with:

groups = engine_id

PHASE 6 — Baselines

Time: 2 days

Don't start with XGBoost.

Establish simple baselines.

Baseline 0

Constant predictor.

Baseline 1

Linear Regression.

Baseline 2

Decision Tree.

Baseline 3

Random Forest.

Record:

MAE

RMSE

R^2

NASA Score

Training time

PHASE 7 — Strong classical models

Time: 2–3 days

Train:

XGBoost
LightGBM
CatBoost

But you don't need to spend equal effort on all three.

I'd focus primarily on:

XGBoost

and perhaps LightGBM as comparison.

Tune parameters such as:

n_estimators
max_depth
learning_rate
subsample
colsample_bytree

Use validation engines, not test data, for model selection.

PHASE 8 — RUL target experiments

A useful experiment is:

Raw RUL

vs

Capped RUL

For example:

$RUL_{target} = \min(RUL, 125)$

This represents an assumption that distinguishing 250 from 220 remaining cycles may not be meaningful when the engine is still essentially healthy.

Treat the cap as a hyperparameter/modeling assumption and compare results rather than presenting 125 as a physical constant.

PHASE 9 — Evaluation

Time: 1–2 days

Use:

MAE

$$\text{MAE} = \frac{1}{N} \sum |y - \hat{y}| \quad \text{MAE} = \frac{1}{N} \sum |y - \hat{y}| \quad \text{MAE} = \frac{1}{N} \sum |y - \hat{y}|$$

Easy to interpret.

RMSE

Penalizes larger errors more.

R²

Useful supplementary measure.

NASA Score

This is essential for this project.

The uploaded paper uses an asymmetric scoring function because late RUL prediction is considered worse than early prediction in this prognostics setting.

Suppose:

True RUL = 20

Predict:

15

You're conservative.

Predict:

40

You're saying the engine has much longer left than it actually does.

That second error can be more dangerous.

Your model table should eventually look like:

Model	MAE	RMSE	R ²	NASA Score
Linear Regression				
Decision Tree				
Random Forest				
XGBoost				
LightGBM				

PHASE 10 — Error analysis

Don't stop after metrics.

Plot:

Predicted RUL vs True RUL

Residual vs True RUL

Error distribution

Prediction vs truth for individual engines

Then separately evaluate:

RUL > 100

50 < RUL ≤ 100

20 < RUL ≤ 50

RUL ≤ 20

The last group is especially important operationally.

Ask:

Does my model perform well when the engine is actually close to failure?

PHASE 11 — LSTM

Time: 3–4 days

Now switch from:

engineered tabular features

to:

sensor sequences

Create sliding windows.

For sequence length 50:

Engine #4

cycles 1 → 50



LSTM



RUL at cycle 50

cycles 2 → 51



LSTM



RUL at cycle 51

Your tensor:

X

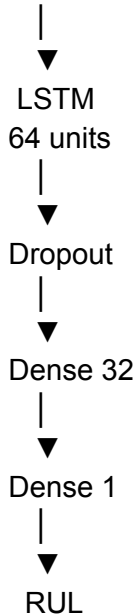
samples × 50 × features

Never let a window cross engine boundaries.

Step 11.1 — Simple architecture

Start with:

Sensor Sequence



Use PyTorch.

Try:

Adam
Huber/MSE loss
early stopping
learning-rate scheduling

Keep the first model simple.

PHASE 12 — GRU

Implement the same architecture with GRU.

Compare:

LSTM

vs

GRU

on:

RMSE

NASA score

training time

model size

inference time

This is a much more useful experiment than simply saying you implemented both.

PHASE 13 — Transformer / TCN

Only do this **after the entire MVP works**.

Try one of:

Transformer Encoder

or

Temporal Convolutional Network

You don't need both.

For placements, I'd rather see:

deployed XGBoost + LSTM system

than:

six unfinished neural architectures.

PHASE 14 — Select the production model

Don't automatically choose whichever model sounds fanciest.



Local explanation

Much more interesting:

Engine #15

Current cycle = 175

Predicted RUL = 18

Then:

Feature	effect
sensor_11 trend	↓ RUL
sensor_4 rolling mean	↓ RUL
sensor_7	↓ RUL
operating condition	↑ RUL

Now the system explains **why** it thinks the engine is approaching failure.

PHASE 16 — Health Score

You want:

Health

84 / 100

There are two sensible approaches.

Simple MVP

Derive it transparently from predicted RUL:

$$\text{Health} = 100 \min\left(1, \frac{\widehat{\text{RUL}}}{\text{RUL}_{\text{healthy}}}\right)$$
$$\text{Health} = 100 \min\left(1, \frac{\text{RUL}}{\text{RUL}_{\text{healthy}}}\right)$$

This is an application-level health score.

Advanced

Learn a degradation representation from sensor trajectories using:

PCA

or

Autoencoder

But don't call either of these the paper's true health index. The paper's health index was calculated from normalized operational margins and was not explicitly exposed to participants.

PHASE 17 — Failure probability

This is one of the extensions I most strongly recommend.

Create classification targets:

```
df["fail_10"] = (df["RUL"] <= 10).astype(int)
df["fail_20"] = (df["RUL"] <= 20).astype(int)
df["fail_30"] = (df["RUL"] <= 30).astype(int)
```

Then predict:

Engine #45

Failure within 10 cycles
32%

Failure within 20 cycles
81%

Failure within 30 cycles
94%

This is often easier for maintenance personnel to interpret than RUL alone.

Evaluate using:

Precision
Recall

F1
PR-AUC
ROC-AUC

PHASE 18 — Prediction uncertainty

Do **not** display:

Confidence = 91%

unless you've actually calculated something meaningful.

RUL regression doesn't naturally give you a probability.

A better dashboard feature is:

Predicted RUL

18 cycles

Prediction interval

13–25 cycles

Later implement:

Conformal prediction

or quantile regression.

This is a strong advanced ML addition.

PHASE 19 — Anomaly detection

This is optional.

Start with:

Isolation Forest

rather than immediately building an autoencoder.

Train primarily around healthy/early-life behavior.

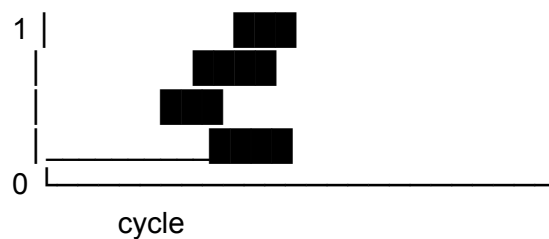
Then calculate:

Anomaly Score

over an engine's life.

Ideally:

Anomaly
score



Investigate whether anomaly score rises before failure.

PHASE 20 — Similar engine search

Optional, but visually impressive.

For:

Engine #45

find historical engines with similar degradation trajectories.

Start simple with engineered representations.

Later use:

Dynamic Time Warping

Display:

Current Engine #45

Similar engines:

#18

#63

#72

Then overlay their sensor/health trajectories.

PHASE 21 — What-if analysis

Rename this feature:

Model Sensitivity Analysis

Suppose:

sensor_11

-5%

Current

+5%

Then calculate:

Predicted RUL

27

21

14

This tells users how sensitive the ML prediction is to a sensor.

It **does not prove** that physically changing that sensor measurement would increase engine life, so don't present it as a causal maintenance simulation.

PHASE 22 — Refactor notebooks into production code

Once experiments are finished:

notebooks

become documentation/research.

Production logic goes into:

src/

For example:

```
src/  
|  
├── data_loader.py  
├── preprocessing.py  
├── feature_engineering.py  
├── sequence.py  
├── train.py  
├── evaluate.py  
├── predict.py  
└── health.py
```

Eventually:

```
python -m src.train
```

should train the model without opening a notebook.

PHASE 23 — Save the entire ML pipeline

Don't save only:

model.pkl

You need:

model

scaler

selected sensor list

feature definitions

RUL target configuration

operating-condition preprocessing

because production preprocessing must be identical to training preprocessing.

PHASE 24 — FastAPI

Time: 2–3 days

Create:

GET /health

POST /predict

POST /predict-file

POST /failure-risk

POST /explain

For example:

```
{  
  "engine_id": 45,  
  "predicted_rul": 18.2,  
  "health_score": 27,  
  "risk": "HIGH",  
  "failure_probability_20": 0.87  
}
```

Important API design issue

Don't accept just:

one sensor reading

if your production model requires history.

For LSTM:

last 50 cycles

may be required.

For XGBoost using rolling features:

last 20 cycles

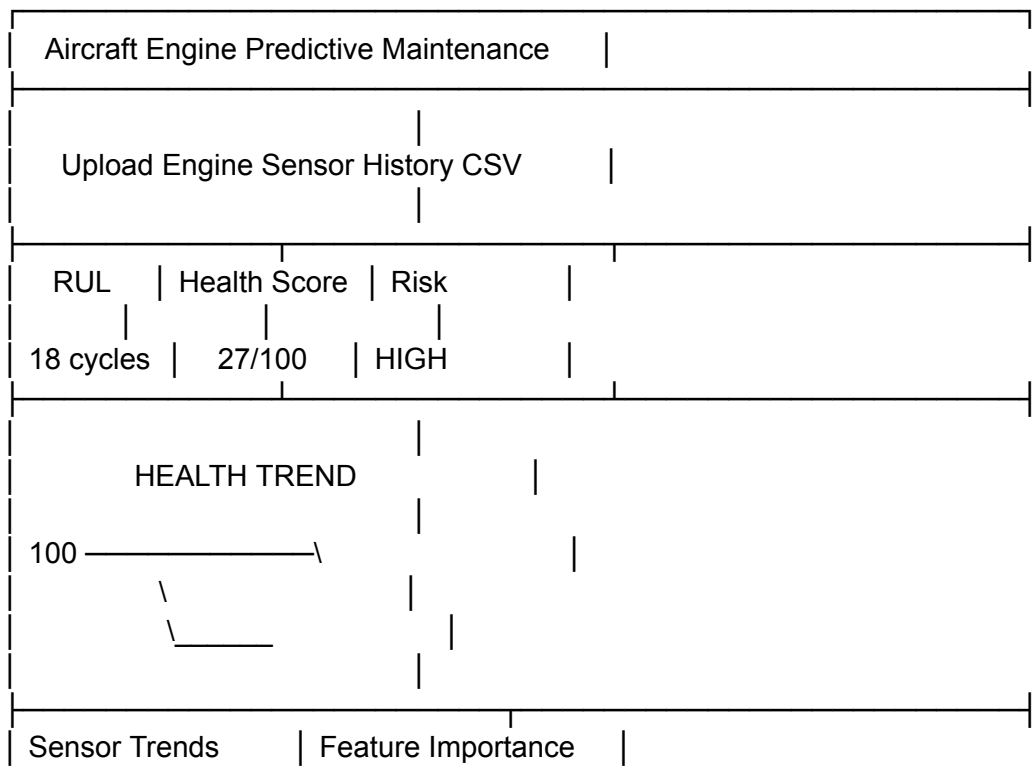
might be required.

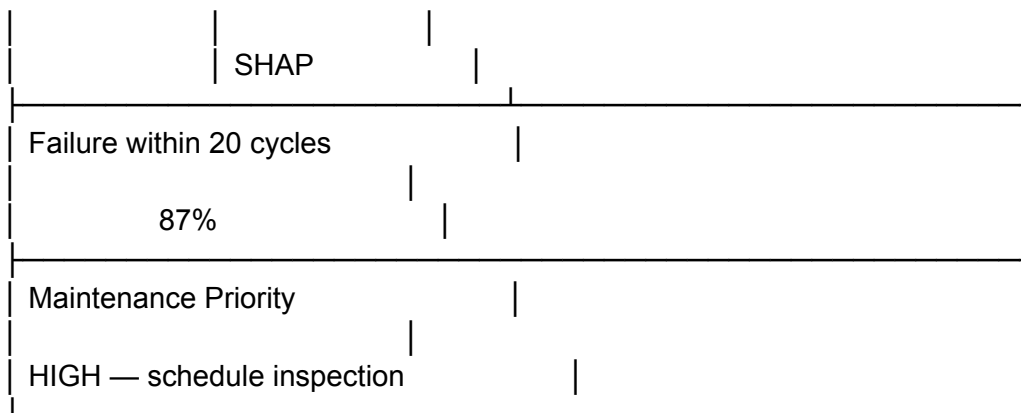
So the backend should accept an **engine sensor history**.

PHASE 25 — React dashboard

Time: 4–5 days

The main dashboard could look like:





Use Plotly for interactive sensor trends.

PHASE 26 — Maintenance recommendations

Be conservative here.

The dataset does not give you enough information to confidently say:

“Replace turbine blade.”

Instead:

RUL > 60

LOW

Continue monitoring

$30 < \text{RUL} \leq 60$

MEDIUM

Increase inspection frequency

$\text{RUL} \leq 30$

HIGH

Schedule maintenance inspection

These are your **application decision rules**, not NASA-prescribed maintenance thresholds.

Clearly state that.

PHASE 27 — Tests

Write unit tests for:

RUL calculation
data loading
scaling
lag generation
rolling features
sequence creation
model loading
prediction schema
API validation

One particularly important test:

Sequence windows NEVER cross
engine boundaries.

Another:

No future observations are used
to construct current features.

PHASE 28 — Docker

Containerize the application.

Eventually:

docker compose up

should launch:

React

|

